```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    classification_report, confusion_matrix,
    roc_auc_score, roc_curve,
    precision_recall_curve, f1_score
)

%matplotlib inline

train = pd.read_csv('../data/processed/training_xgb_rf.csv')
test  = pd.read_csv('../data/processed/test_xgb_rf.csv')

X_train = train.drop(columns=['label'])
y_train = train['label']
X_test  = test.drop(columns=['label'])
y_test  = test['label']

print(f'Train: {X_train.shape}, Test: {X_test.shape}')
print(f'Class balance (train) — 0: {(y_train==0).sum()}, 1:
{(y_train==1).sum()}')
```

```
Train: (107740, 58), Test: (55945, 58)
Class balance (train) — 0: 51890, 1: 55850
```

```python
base_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)
```

Hyperparameter Tuning

```python
from sklearn.model_selection import RandomizedSearchCV

param_dist = {
    'n_estimators':      [100, 200, 300, 500],
    'max_depth':         [None, 10, 20, 30, 40],
    'min_samples_split': [2, 5, 10, 20],
    'min_samples_leaf':  [1, 2, 4, 8],
    'max_features':      ['sqrt', 'log2', 0.3, 0.5],
    'bootstrap':         [True, False],
}

search = RandomizedSearchCV(
    base_model,
    param_distributions=param_dist,
```

```
    n_iter=30,
    scoring='roc_auc',
    cv=3,
    n_jobs=-1,
    random_state=42,
    verbose=1
)

search.fit(X_train, y_train)

print(f'Best ROC-AUC (CV): {search.best_score_:.4f}')
print(f'Best params:       {search.best_params_}')
```

```
Fitting 3 folds for each of 30 candidates, totalling 90 fits
Best ROC-AUC (CV): 0.9484
Best params:       {'n_estimators': 300, 'min_samples_split': 20,
'min_samples_leaf': 4, 'max_features': 0.3, 'max_depth': 10,
'bootstrap': True}
```

```
model = search.best_estimator_

model.fit(X_train, y_train)
```

```
RandomForestClassifier(max_depth=10, max_features=0.3,
min_samples_leaf=4,
                       min_samples_split=20, n_estimators=300,
n_jobs=-1,
                       random_state=42)
```

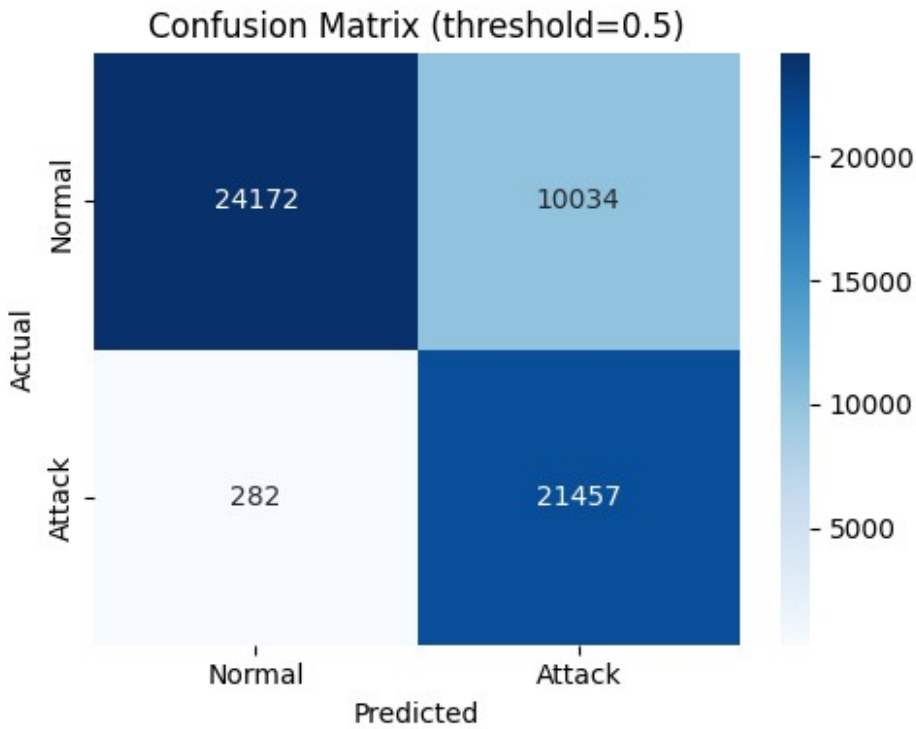
Base Evaluation, threshold = 0.5

```
y_prob = model.predict_proba(X_test)[:, 1]
y_pred = (y_prob >= 0.5).astype(int)

print(classification_report(y_test, y_pred, target_names=['Normal',
'Attack']))
print(f'ROC-AUC: {roc_auc_score(y_test, y_prob):.4f}')
```

```
              precision    recall  f1-score   support

      Normal       0.99      0.71      0.82     34206
      Attack       0.68      0.99      0.81     21739

    accuracy                           0.82     55945
   macro avg       0.83      0.85      0.82     55945
weighted avg       0.87      0.82      0.82     55945

ROC-AUC: 0.9729
```

```
cm = confusion_matrix(y_test, y_pred)
fig, ax = plt.subplots(figsize=(5, 4))
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
            xticklabels=['Normal', 'Attack'], yticklabels=['Normal',
'Attack'])
ax.set_ylabel('Actual')
ax.set_xlabel('Predicted')
ax.set_title('Confusion Matrix (threshold=0.5)')
plt.tight_layout()
plt.show()
```



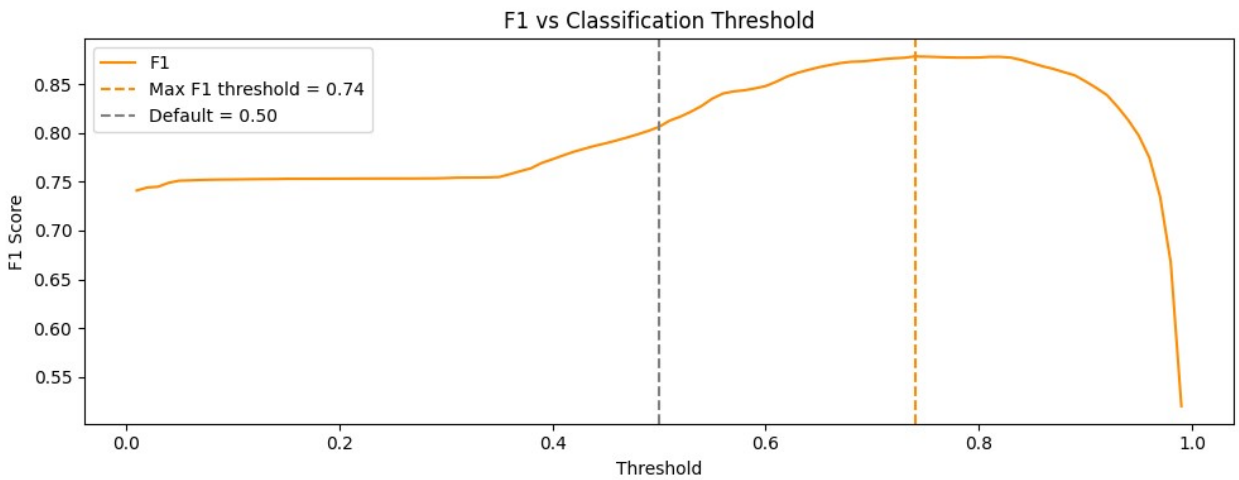
Threshold Tuning

In cybersecurity, false negatives (missed attacks) are more costly than false positives. We find the threshold that maximises F1, then also examine the recall-precision tradeoff.

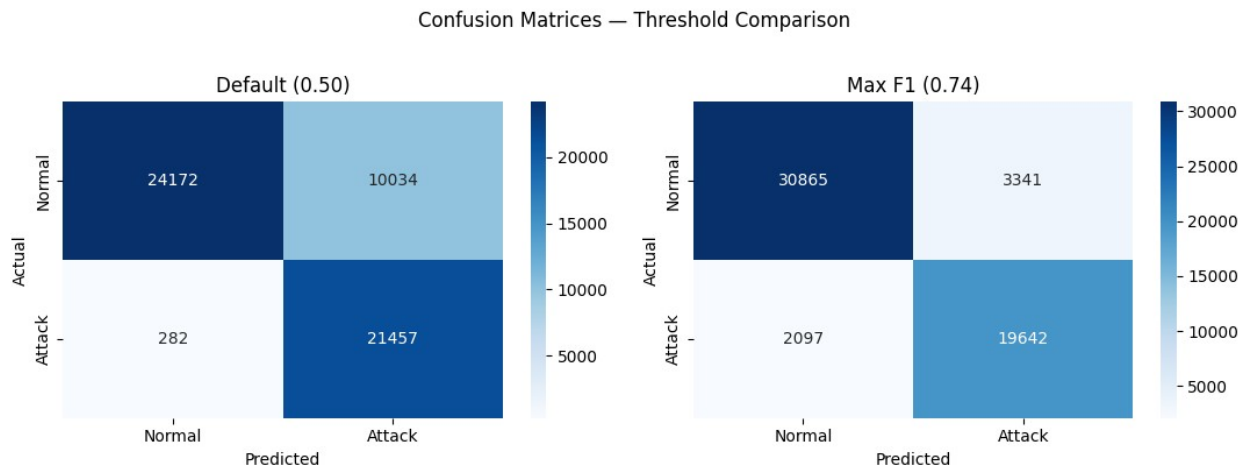
```
thresholds = np.arange(0.01, 1.0, 0.01)
f1_scores  = [f1_score(y_test, (y_prob >= t).astype(int)) for t in
thresholds]

best_f1_threshold = thresholds[np.argmax(f1_scores)]

print(f'Optimal threshold (max F1): {best_f1_threshold:.2f}  — F1:
{max(f1_scores):.4f}')
print(f'Default threshold (0.5):           — F1: {f1_scores[49]:.4f}')

fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(thresholds, f1_scores, color='darkorange', label='F1')
```

```
ax.axvline(best_f1_threshold, color='darkorange', linestyle='--',
label=f'Max F1 threshold = {best_f1_threshold:.2f}')
ax.axvline(0.5, color='gray', linestyle='--', label='Default = 0.50')
ax.set_xlabel('Threshold')
ax.set_ylabel('F1 Score')
ax.set_title('F1 vs Classification Threshold')
ax.legend()
plt.tight_layout()
plt.show()
```

```
Optimal threshold (max F1): 0.74  — F1: 0.8784
Default threshold (0.5):           — F1: 0.8062
```




```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
for ax, (t, title) in zip(axes, [
    (0.5,                 'Default (0.50)'),
    (best_f1_threshold, f'Max F1 ({best_f1_threshold:.2f})'),
]):
    preds = (y_prob >= t).astype(int)
    sns.heatmap(confusion_matrix(y_test, preds), annot=True, fmt='d',
cmap='Blues', ax=ax,
                xticklabels=['Normal', 'Attack'],
yticklabels=['Normal', 'Attack'])
    ax.set_ylabel('Actual')
    ax.set_xlabel('Predicted')
    ax.set_title(title)
plt.suptitle('Confusion Matrices — Threshold Comparison', y=1.02)
plt.tight_layout()
plt.show()

print(f'\nChosen threshold: {best_f1_threshold:.2f} (max F1)')
print(classification_report(y_test, (y_prob >=
best_f1_threshold).astype(int), target_names=['Normal', 'Attack']))
```




```
Chosen threshold: 0.74 (max F1)
              precision    recall  f1-score   support

      Normal       0.94      0.90      0.92     34206
      Attack       0.85      0.90      0.88     21739

    accuracy                           0.90     55945
   macro avg       0.90      0.90      0.90     55945
weighted avg       0.90      0.90      0.90     55945
```

```
y_pred_tuned = (y_prob >= best_f1_threshold).astype(int)
```

Per-Attack-Category Breakdown

Join predictions against the raw test data to evaluate performance per attack type.

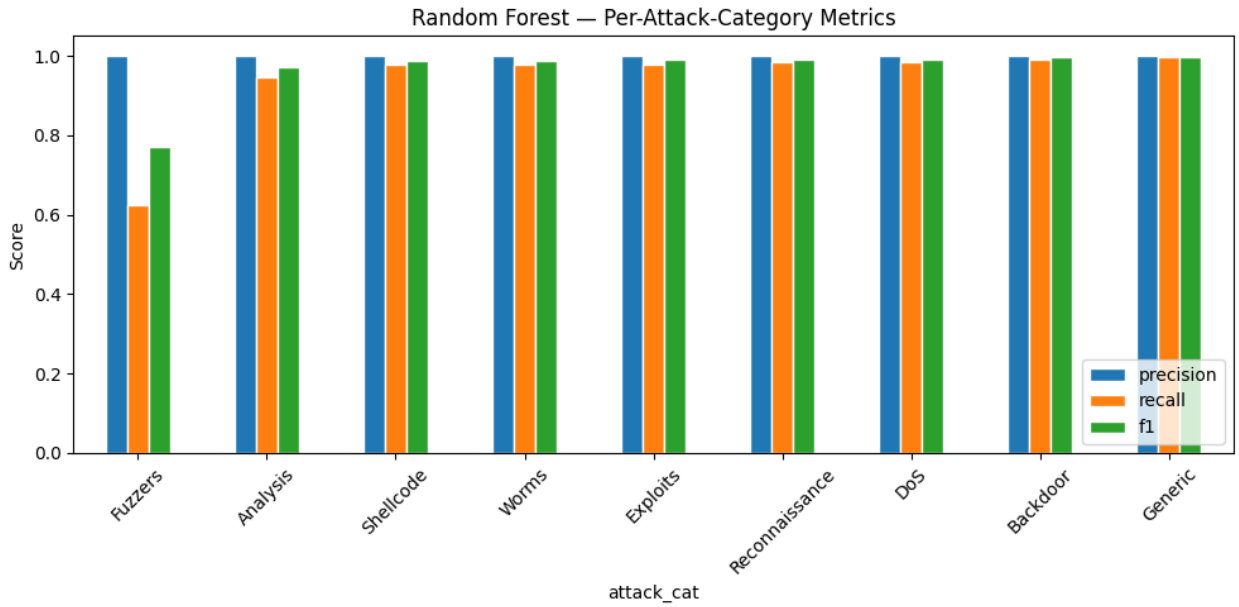
```
raw_test = pd.read_csv('../data/raw/UNSW_NB15_testing-set.csv')
raw_test = raw_test.drop_duplicates(subset=[c for c in
raw_test.columns if c != 'id']).reset_index(drop=True)

results = raw_test[['attack_cat']].copy()
results['y_true'] = y_test.values
results['y_pred'] = y_pred_tuned
results['y_prob'] = y_prob

results.head()
```

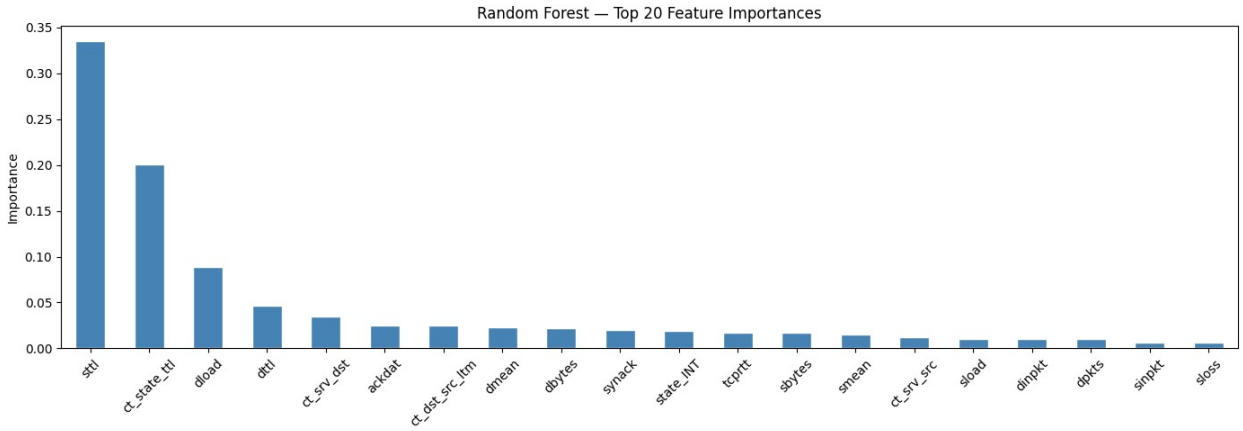
```
  attack_cat  y_true  y_pred    y_prob
0     Normal       0       1  0.852280
1     Normal       0       1  0.822710
2     Normal       0       1  0.760445
3     Normal       0       0  0.736482
4     Normal       0       1  0.750704
```

```
def attack_metrics(group):
    tp = ((group['y_true'] == 1) & (group['y_pred'] == 1)).sum()
    fn = ((group['y_true'] == 1) & (group['y_pred'] == 0)).sum()
    fp = ((group['y_true'] == 0) & (group['y_pred'] == 1)).sum()
    tn = ((group['y_true'] == 0) & (group['y_pred'] == 0)).sum()
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0
    recall    = tp / (tp + fn) if (tp + fn) > 0 else 0
    f1        = 2 * precision * recall / (precision + recall) if
(precision + recall) > 0 else 0
    return pd.Series({'count': len(group), 'precision':
round(precision, 4),
                      'recall': round(recall, 4), 'f1': round(f1, 4)})

attack_results = results[results['attack_cat'] != 'Normal']
breakdown = attack_results.groupby('attack_cat').apply(attack_metrics,
include_groups=False).sort_values('recall')
print(breakdown.to_string())
```

```
                  count  precision  recall      f1
attack_cat
Fuzzers          4838.0        1.0  0.6255  0.7696
Analysis          446.0        1.0  0.9462  0.9724
Shellcode         378.0        1.0  0.9762  0.9880
Worms              44.0        1.0  0.9773  0.9885
Exploits         7609.0        1.0  0.9787  0.9892
Reconnaissance   2703.0        1.0  0.9834  0.9916
DoS              1718.0        1.0  0.9837  0.9918
Backdoor          346.0        1.0  0.9913  0.9956
Generic          3657.0        1.0  0.9964  0.9982
```

```
fig, ax = plt.subplots(figsize=(10, 5))
breakdown[['precision', 'recall', 'f1']].plot(kind='bar', ax=ax,
edgecolor='white')
ax.set_title('Random Forest — Per-Attack-Category Metrics')
ax.set_ylabel('Score')
ax.set_ylim(0, 1.05)
ax.tick_params(axis='x', rotation=45)
ax.legend(loc='lower right')
plt.tight_layout()
plt.show()
```



Feature Importance

```
importances = pd.Series(model.feature_importances_,
index=X_train.columns).sort_values(ascending=False)

fig, ax = plt.subplots(figsize=(14, 5))
importances.head(20).plot(kind='bar', ax=ax, color='steelblue',
edgecolor='white')
ax.set_title('Random Forest — Top 20 Feature Importances')
ax.set_ylabel('Importance')
ax.tick_params(axis='x', rotation=45)
plt.tight_layout()
plt.show()
```